Séquence 1

1

Pour aller plus loin — Ce qui se passe derrière la connexion

Ce complément développe les notions de S1 et prépare les questions qui seront reprises en S2 à S5. Il reste disponible comme ressource.

1. Connexion, message et trame : trois échelles

La connexion est le canal établi entre deux extrémités. Le message est une unité de données remise à l'application. Une trame est une unité du protocole qui transporte des données ou du contrôle. Un message peut être réparti sur plusieurs trames : une trame n'est donc pas nécessairement une bulle de chat. Le navigateur et la bibliothèque serveur s'occupent de ce niveau.

En apprendre + : Échanges de trames — MDN

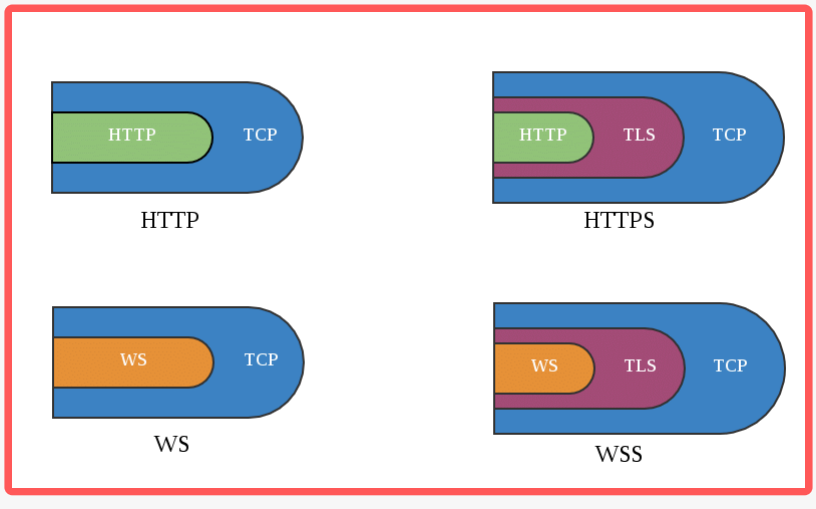
Pour raisonner sur notre chat, pensez d'abord « qui envoie quel message à qui ? ». Vous n'aurez pas à construire manuellement des trames ni à compter leurs octets pour réussir les exercices.

Le protocole accepte du texte et du binaire. Dans notre projet, nous choisirons des messages textuels structurés en S3. Le format de ces messages relève de l'application : le protocole WebSocket ne connaît pas spontanément un auteur, un salon ou une confirmation.

En apprendre + : Référence WebSocket — MDN

2. Où placer TCP et TLS ?

Dans le cas classique étudié, WebSocket repose sur TCP. TCP fournit un transport fiable et ordonné tant que la connexion permet les échanges ; cela ne garantit pas qu'une application destinataire a traité ou enregistré ce qu'elle a reçu. Avec wss://, TLS protège le transport par chiffrement. ws:// désigne la variante sans TLS.



3. Pourquoi commence-t-on par HTTP ?

L'ouverture HTTP/1.1 utilise le mécanisme Upgrade. Le client demande le protocole WebSocket et le serveur qui l'accepte répond 101 Switching Protocols. La connexion change alors d'usage. Une réponse 200 OK sur /health prouve qu'un service HTTP répond, mais n'est pas une validation du handshake WebSocket.

En apprendre **+** *: Mécanisme Upgrade — MDN*

Le chemin /chat est le point d'entrée choisi pour notre application. Ce nom n'a pas une signification imposée par WebSocket. NestJS écoutera ici sur le même port 3000 pour HTTP et WebSocket. Il n'est pas nécessaire d'avoir un port dédié pour chaque protocole.

Le handshake complet comprend aussi des en-têtes spécifiques. Vous les observerez en S2 ; vous n'aurez pas à les fabriquer dans le frontend, car le constructeur natif s'en charge. Nous ne présentons pas 101 comme le mécanisme universel de toutes les versions de HTTP : c'est le cas HTTP/1.1 de l'atelier.

4. Que signifie « la connexion est ouverte » ?

L'API distingue quatre états : CONNECTING, OPEN, CLOSING et CLOSED. Créer un objet WebSocket ne signifie pas que la connexion est immédiatement utilisable. L'application doit attendre son ouverture avant d'envoyer. Une fois fermée, la connexion ne redevient pas ouverte : une reprise crée une nouvelle connexion.

En apprendre **+** *: Propriété readyState — MDN*

Ces états répondent à une question technique : « Puis-je utiliser ce canal ? ». Les statuts d'un message répondent à une autre question : « Mon envoi a-t-il été traité ? ». En S5, l'interface devra gérer ces deux informations séparément.

Exemple : le badge peut afficher « connecté » tandis qu'un message est encore « en attente de confirmation ». Il n'y a aucune contradiction : le canal fonctionne, mais la réponse métier n'est pas encore arrivée.

5. Pourquoi prévoir des confirmations si TCP est fiable ?

Imaginez cette chronologie :

1. Camille demande l'envoi de « Bonjour ».
2. Le serveur reçoit les données et commence à les traiter.
3. La connexion coupe avant que Camille reçoive le résultat.

Camille ne peut pas déduire de la coupure si le serveur a terminé le traitement. Réessayer sans précaution peut produire un doublon ; ne pas réessayer peut laisser une incertitude. Notre S5 introduira un identifiant permettant de relier une confirmation à l'envoi concerné. L'absence de confirmation sera affichée comme « non confirmé », pas comme une preuve certaine de non-réception.

De même, « confirmé par le serveur » et « lu par Alex » sont deux informations différentes. Nous implémenterons la première. Un accusé de lecture demanderait un échange supplémentaire défini par l'application.

6. Maintenir un canal a aussi un coût

Chaque connexion nécessite des ressources côté serveur. Il faut savoir quelles connexions sont ouvertes et retirer celles qui sont fermées. Si les envois s'accumulent plus vite que leur transmission, des données restent en attente ; la propriété bufferedAmount du navigateur rend visible ce volume. Ce sujet apparaîtra en complément de S4.

En apprendre **+** *:* API WebSocket et données en attente — *MDN*

Une connexion persistante n'est donc pas « gratuite ». Son intérêt pour le chat est d'éviter le sondage périodique et de permettre les échanges dès que les événements surviennent. Il faut comparer cet avantage aux besoins réels : un tableau d'information actualisé une fois par jour n'a pas le même besoin qu'une conversation active.

7. Une coupure n'est pas toujours immédiatement visible

Fermer volontairement une connexion et perdre brutalement le réseau sont deux situations différentes. Un onglet peut encore croire sa connexion utilisable alors que le chemin réseau ne fonctionne plus. Le mécanisme de contrôle ping/pong peut aider le serveur à détecter les connexions qui ne répondent plus. Il sera montré en S5. Les trames ping/pong du protocole ne sont pas des commandes JavaScript ping() et pong() exposées par l'API WebSocket du navigateur.

En apprendre **+** *:* Ping/pong — *MDN*

La règle métier est tout aussi importante : si l'utilisateur clique volontairement sur « Déconnecter », faut-il rouvrir immédiatement ? Non dans notre parcours. La reconnexion doit distinguer la fermeture voulue de l'incident et éviter de créer plusieurs tentatives concurrentes.

8. Pourquoi ne pas prendre une bibliothèque qui fait tout ?

Le choix est pédagogique : nous voulons comprendre et programmer les mécanismes nécessaires à la messagerie. Utiliser l'API native rend visibles la connexion, les messages et les fermetures. Nous construirons ensuite les comportements de reconnexion, de confirmation et de salon.

NestJS et ws nous évitent d'écrire un serveur de protocole bas niveau. Ils ne remplacent pas les décisions sur les destinataires et les règles du projet. Les identifiants de connexion, le registre des clients et le salon actif seront gérés par notre code.

En S3, le guide formateur précisera la forme de message attendue par WsAdapter. C'est une convention du framework pour trouver le bon gestionnaire, pas une obligation du protocole WebSocket. Il est utile de conserver cette séparation : le protocole transporte ; l'application donne un sens aux données.